

SIH26184 Engineering Blueprint – Predictive Cybercrime Intelligence Platform

SIH26184 | Cybercrime Predictive Intelligence Platform

SMART INDIA HACKATHON 2026

Problem Statement: Development of a Predictive Analytics Framework for Cybercrime Complaints to Forecast Likely Cash Withdrawal Locations in Advance, Enabling Generation of Actionable Intelligence for Timely and Proactive Cybercrime Intervention.

Document Type: Software Solution | Engineering & Product Blueprint

Team Name:

Version: 1.0 | Date: September 2026

Document Purpose

This document is the team's single source of truth for product scope, architecture, data/ML approach, technology choices, collaboration workflow, testing, security, deployment, and demo strategy. It is intentionally written as an engineering blueprint rather than a final academic report.

Table of Contents

1. Executive Summary
2. Problem Definition
3. Objectives
4. Users and Stakeholders
5. Scope and Boundaries
6. Proposed Solution
7. Core Use Cases
8. Functional Requirements
9. Non-Functional Requirements
10. Product and System Workflow
11. System Architecture
12. Data Architecture
13. Predictive Analytics and Intelligence Methodology
14. Risk Scoring and Explainability

15. Application Modules
16. GIS and Investigation Experience
17. Alert and Notification System
18. Technology Stack
19. API and Integration Contract
20. Development and Collaboration Workflow
21. Repository and Code Organization
22. Team Responsibilities
23. Testing and Evaluation Strategy
24. Security, Privacy and Governance
25. MLOps and Feedback Loop
26. Deployment Architecture
27. MVP and Future Scope
28. Risks, Assumptions and Mitigations
29. Demo Workflow
30. Deliverables and Presentation Plan
31. Definition of Done
32. Reference Standards and Engineering Principles

1. Executive Summary

The proposed solution is a predictive cybercrime intelligence platform designed to help law-enforcement and financial-sector investigators identify likely future cash-withdrawal hotspots associated with cybercrime incidents. The system combines complaint information, transaction patterns, temporal behavior, geospatial features, and entity/network relationships to produce ranked predictions of likely cash-out locations and time windows.

The platform is designed as decision-support software. It does not autonomously apprehend suspects, freeze accounts, or take irreversible operational action. Instead, it converts data and predictive outputs into explainable, auditable intelligence that authorized personnel can review and act upon.

Core Value Proposition

From reactive investigation to proactive intervention: identify where and when illicit funds are likely to be withdrawn, explain why the location is risky, and place that intelligence in the investigator's workflow before the expected cash-out window.

2. Problem Definition

Cybercrime investigations often begin after a fraudulent transaction has already occurred. The challenge addressed by this problem statement is to make the investigation more proactive by forecasting likely locations where illicit funds may be converted into cash.

Primary Question

Given the current complaint and associated transaction/network information, which locations are most likely to be used for cash withdrawal within a defined future time window?

Problem Characteristics

- The signal is distributed across complaints, transactions, time and geography.
- Multiple accounts or intermediaries may exist between a victim transaction and a cash-out event.
- Predictions are probabilistic, not certain, and must therefore include confidence and explanation.
- Operational users need prioritized locations, not just raw model scores.
- Sensitive data requires authentication, authorization, auditability and privacy controls.

3. Objectives

- Forecast likely cash-withdrawal locations associated with cybercrime cases before the withdrawal occurs.
- Estimate a useful future time window along with each location prediction.
- Combine temporal, spatial, transactional and network signals into a unified risk assessment.
- Provide investigators with GIS-based visualization, drill-down investigation tools and explainable risk factors.
- Generate prioritized alerts for authorized users based on configurable risk thresholds.
- Record outcomes and investigator feedback so that model and operational performance can be evaluated and improved.

4. Users and Stakeholders

User	Primary Need	Key Capabilities
Government / Law Enforcement User	Investigate active cybercrime cases, monitor predictive cash-out hotspots, and act on actionable intelligence	Case management, risk heatmaps, predictive analysis, network visualization, alert review, and evidence recording
System Administrator	Operate, configure, and secure the platform and user access control	User and RBAC management, system configuration, model versioning, and security audit logs

5. Scope and Boundaries

5.1 In Scope for MVP

- Complaint and case management using structured or synthetic data.
- Transaction and account relationship analysis.
- ATM/location data and geospatial processing.
- Prediction of top likely cash-out locations and a defined prediction window.
- Risk score, confidence and explanation.
- GIS risk heatmap and location drill-down.
- Investigator case view and network visualization.
- Dashboard-based alert generation and acknowledgement.
- Role-based authentication and audit logging.

5.2 Out of Scope for MVP

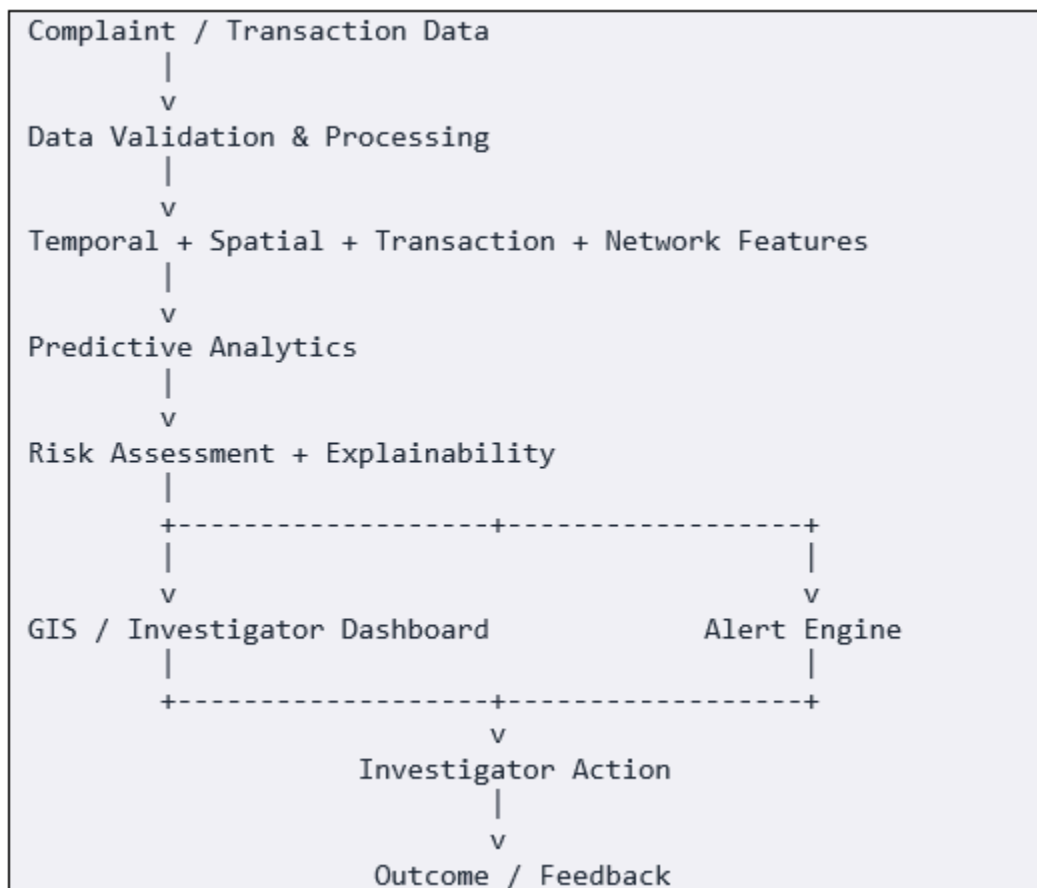
- Direct access to confidential banking/NCRP/I4C systems unless officially provided.
- Autonomous account freezing, debit liens or legal orders.
- Autonomous police deployment or suspect apprehension decisions.
- Production-grade cross-border intelligence exchange.
- Large-scale real-time streaming infrastructure beyond prototype needs.

5.3 Future Scope

- Direct institutional API integrations subject to authorization.
- AePS/CSP and merchant/POS extensions.
- Streaming event ingestion.
- Advanced graph ML / graph neural networks if justified by data volume and performance.
- Automated model monitoring and scheduled retraining.
- Multi-state intelligence coordination features.

6. Proposed Solution

The solution is a modular web-based intelligence platform. It separates the user interface, application services, data storage, predictive intelligence, and operational alerting so that each layer can evolve independently while sharing stable interfaces.



7. Core Use Cases

Use Case	Description
UC-01: Predictive ATM Cash-Out Risk (MVP)	Predict top ATM/ATM-cluster locations likely to be used for withdrawal within the next N hours.
UC-02: Micro-ATM / AePS Risk Detection	Identify unusual withdrawal concentration at geographically distributed service points; initially implemented as an extension of the same spatial-risk pipeline.
UC-03: Mule Account Network Analysis	Trace related accounts and transactions to identify suspicious intermediaries and network concentration.
UC-04: Cross-Regional Pattern Detection	Aggregate cases across areas to surface linked locations, entities and recurring patterns.
UC-05: High-Value Merchant/POS Risk	Future extension to suspicious localized spending where merchant data is available.

8. Functional Requirements

Authentication & Authorization

- Login/logout
- Role-based access control
- Session/token handling
- Unauthorized request rejection

Case Management

- Create/update/view cases
- Search and filter complaints

- Link transactions/entities to cases
- Track investigation status

Prediction

- Run prediction for a case
- Return top-K likely locations
- Return risk score and confidence
- Return future time window
- Store prediction version and timestamp

GIS

- Display risk zones
- Show ATM/location markers
- Filter by region/time/category
- Open location details
- Display prediction evidence

Investigation

- View transaction history
- View network relationships
- Review explanation
- Record investigator notes/actions
- Track case/alert status

Alerts

- Create alert from risk threshold
- Assign severity
- Acknowledge/resolve
- Maintain alert history
- Support notification adapters for future integrations

9. Non-Functional Requirements

Area	Requirement
Security	Authentication, authorization, encrypted transport, secure secrets, audit logs, input validation, dependency hygiene.
Performance	Define target response times for dashboard/API/prediction operations; keep GIS interactions responsive for expected prototype load.
Reliability	Graceful API errors, retries where appropriate, data validation, backups for persisted project data.
Scalability	Modular services, indexed geospatial data, containerized deployment and stateless API design where possible.
Maintainability	Typed interfaces, linting, tests, documentation, code review and predictable folder structure.
Observability	Structured logs, request IDs, prediction logs and basic health checks.
Explainability	Every operational prediction should expose the main contributing factors and model/version metadata.

10. Product and System Workflow

10.1 Product Workflow

New complaint / transaction signal -> Case created or linked -> Relevant data assembled -> Temporal, spatial and network analysis -> Prediction generated -> Risk assessed -> GIS and

investigator view updated -> Alert generated when threshold is met -> Investigator reviews and acts
-> Outcome recorded -> Outcome feeds evaluation / improvement

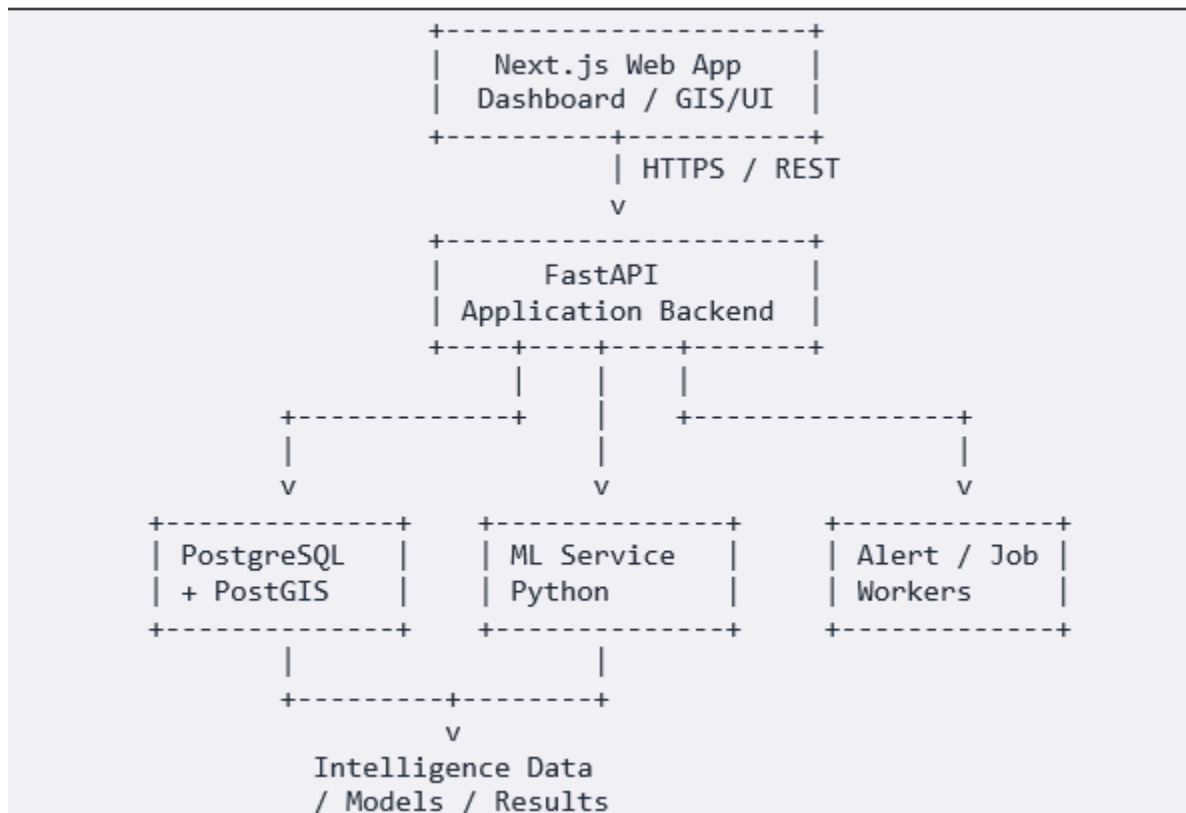
10.2 Development Workflow

Requirements -> UX/Product design -> Architecture -> Data model/API contract -> Parallel
implementation -> Integration -> Testing -> Security review -> CI/CD -> Deployment -> Monitoring
-> Feedback -> Iteration

10.3 Team Collaboration Workflow

Issue -> Feature branch -> Implementation -> Local tests -> Pull Request -> Code review -> CI
checks -> Merge -> Integration -> Demo

11. System Architecture



Architecture Decision

Use a modular monolith for the business backend plus a separate Python ML service. Avoid microservices unless a demonstrated scaling or ownership requirement appears. For a six-person SIH team, minimizing distributed-system overhead is more valuable than creating enterprise-looking complexity.

12. Data Architecture

12.1 Data Domains

Domain	Representative Fields
Cases	<code>case_id</code> , <code>type</code> , <code>amount</code> , <code>status</code> , <code>created_at</code> , <code>location</code>
Complaints	<code>complaint_id</code> , <code>case_id</code> , <code>reported_at</code> , <code>category</code> , <code>victim_location</code>
Accounts / Entities	<code>entity_id</code> , <code>type</code> , <code>masked_identifier</code> , <code>risk_metadata</code>
Transactions	<code>transaction_id</code> , <code>sender_entity</code> , <code>receiver_entity</code> , <code>amount</code> , <code>timestamp</code> , <code>channel</code>
ATMs / Locations	<code>location_id</code> , <code>type</code> , <code>latitude</code> , <code>longitude</code> , <code>region</code> , <code>metadata</code>
Predictions	<code>prediction_id</code> , <code>case_id</code> , <code>model_version</code> , <code>location_id</code> , <code>score</code> , <code>confidence</code> , <code>time_window</code>
Alerts	<code>alert_id</code> , <code>prediction_id</code> , <code>severity</code> , <code>status</code> , <code>created_at</code> , <code>acknowledged_by</code>

Domain	Representative Fields
Audit Logs	actor, action, object, timestamp, request_id

12.2 Data Processing Pipeline

Raw / Imported Data -> Schema validation -> Duplicate detection -> Missing/invalid value handling -> Normalization -> Geospatial validation -> Entity resolution / linkage -> Feature engineering -> Curated analytical dataset

12.3 Prototype Data Policy

Where real operational data is unavailable, the team will use synthetic or sanitized data. The synthetic generator should preserve realistic distributions and relationships without reproducing real personal or financial information.

13. Predictive Analytics and Intelligence Methodology

13.1 Prediction Framing

Frame the prediction as a ranking problem: for a case and prediction window, return the top-K candidate cash-out locations with calibrated risk probabilities rather than a single deterministic location.

13.2 Feature Groups

Feature Group	Examples
Temporal	Hour of day, day of week, recency, transaction velocity, time since last event

Feature Group	Examples
Spatial	Distance to ATM, local fraud density, ATM density, distance from recent activity
Financial	Amount, amount velocity, number of transactions, concentration
Network	Degree, connected entities, hops, suspicious association count, centrality
Case context	Crime type, complaint count, case age, linked entity count

13.3 Modeling Strategy

- Establish a simple baseline first (for example, historical frequency or logistic regression).
- Compare tree-based models such as Random Forest and gradient boosting (for example, XGBoost/LightGBM) using the same evaluation protocol.
- Use time-aware validation so future observations do not leak into training.
- Calibrate probabilities if they are presented as probabilities to investigators.
- Record model version, training dataset/version and evaluation results.

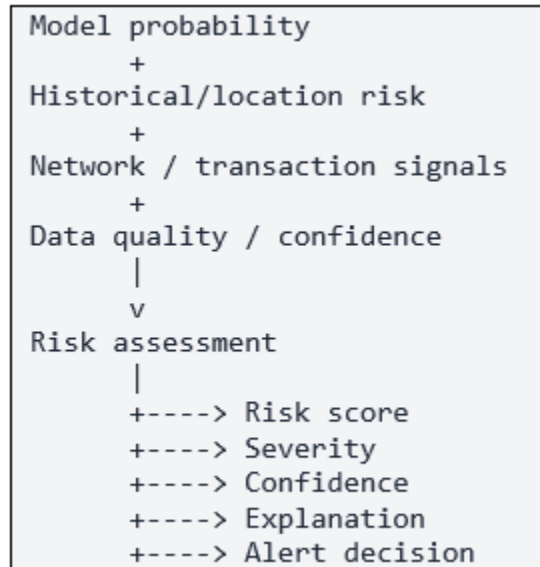
13.4 Model Evaluation

- Top-K hit rate: whether the true location appears in the top K predictions.
- Precision/recall or F1 for the chosen operational threshold.
- PR-AUC/ROC-AUC when appropriate for the classification framing.
- Calibration quality for probability outputs.
- False-positive burden and alert volume.
- Prediction latency and reliability.

Important Note: Do not claim a high real-world accuracy number unless the dataset and evaluation protocol support it. The demo should clearly distinguish synthetic-data performance from operational performance.

14. Risk Scoring and Explainability

The predictive model output should be passed through a risk/decision layer before it becomes an operational alert. This keeps model probability, operational severity and user-facing explanations logically separate.



Explainability Output

- Top contributing features/signals.
- Relevant linked complaints or transaction patterns.
- Spatial/temporal evidence.
- Model version and prediction timestamp.
- Clear wording that the score is a risk estimate, not proof of criminal activity.

15. Application Modules

Module	Purpose
M1. Authentication & RBAC	Secure login, roles, session/token control.
M2. Case & Complaint Management	Create, search, filter and investigate cases.

Module	Purpose
M3. Data & Intelligence Layer	Normalize data, entity relationships and analytical features.
M4. Predictive Analytics	Generate ranked predictions, confidence and time windows.
M5. Risk & Explainability	Convert predictions into actionable severity and evidence.
M6. GIS Intelligence Dashboard	Map, heatmap, filtering and geospatial drill-down.
M7. Investigator Workspace	Case details, network, evidence, notes and actions.
M8. Alert & Notification	Threshold-based alerting, acknowledgement and history.
M9. Audit & Monitoring	System logs, security audit trail, model/prediction logs.

16. GIS and Investigation Experience

16.1 Main Dashboard Layout

+-----+				
	Cybercrime Intelligence Dashboard			
+-----+				
	Active Cases	Critical Risk	Open Alerts	Predictions
+-----+				
	INTERACTIVE RISK MAP			
	red = critical orange = high yellow = medium			
+-----+				
	Filters: Region	Time	Crime Type	Risk Status
+-----+				

16.2 Investigator View Mockup

CASE #10291
Risk: 91% Confidence: High Window: next 6 hours
Likely location: ATM Cluster 14
Why:
- high recent transaction velocity
- multiple related complaints
- nearby suspicious activity
- historical pattern association
[Transactions] [Network] [Map] [Evidence] [Alert]

The UI should distinguish predictions from facts. A predicted hotspot must be visually labeled as a prediction and accompanied by confidence and evidence context.

17. Alert and Notification System

Prediction -> Risk assessment -> Threshold / policy evaluation

- **Low:** dashboard only
- **Medium:** investigator notification
- **High:** priority alert / escalation -> Acknowledge -> Investigate -> Resolve / dismiss -> Record outcome

Notification adapters can be added later for email, SMS or institutional APIs. For the MVP, dashboard notifications and an auditable alert lifecycle are sufficient.

18. Technology Stack

Layer	Recommended Technology	Reason
Web frontend	Next.js + TypeScript	Structured React application, routing, strong type safety

Layer	Recommended Technology	Reason
UI	Tailwind CSS + shadcn/ui	Consistent component system and rapid, maintainable UI work
GIS	MapLibre / Mapbox	Interactive mapping and geospatial visualization
Backend	FastAPI + Python	API development in the same ecosystem as data/ML
Database	PostgreSQL + PostGIS	Relational data plus geospatial queries
ML	Python, pandas, scikit-learn, Random Forest Chosen(Final)	Strong ecosystem for tabular predictive analytics
Graph	NetworkX initially	Simple, transparent graph analytics for the prototype
Cache / jobs	Redis + optional background worker	Useful for asynchronous or expensive operations when needed
API contract	REST + OpenAPI	Clear frontend/backend contract and generated documentation
Version control	Git + GitHub	Collaborative source control, review and CI
Containers	Docker + Docker Compose	Reproducible local development and deployment units
CI/CD	GitHub Actions	Automated tests/build checks on PRs
Testing	Pytest + Playwright + API tests	Backend/unit + browser/e2e coverage

Technology Discipline

Do not add Kafka, Kubernetes, Spark, a vector database, microservices or a graph neural network merely to appear enterprise. Add infrastructure only when a concrete requirement justifies it.

19. API and Integration Contract

The API contract is the boundary that allows frontend, backend, ML and intelligence work to proceed in parallel. The contract should be documented before the teams integrate.

Method	Endpoint	Purpose
POST ▾	/api/auth/login ▾	Authenticate user
GET ▾	/api/dashboard ▾	Dashboard summary
GET ▾	/api/cases ▾	List/filter cases
GET ▾	/api/cases/{id} ▾	Case details
POST ▾	/api/cases ▾	Create case
GET ▾	/api/cases/{id}/transa... ▾	Case transactions
GET ▾	/api/cases/{id}/network ▾	Case network
POST ▾	/api/cases/{id}/predict ▾	Run prediction
GET ▾	/api/predictions/{id} ▾	Prediction details
GET ▾	/api/alerts ▾	List alerts
POST ▾	/api/alerts/{id}/ackno... ▾	Acknowledge alert

Example Prediction Response

```
{  
  
  "prediction_id": "pred_10291",
```

```
"location_id": "atm_cluster_14",

"risk_score": 0.91,

"confidence": "HIGH",

"prediction_window": {

  "start": "14:00",

  "end": "18:00"

},

"model_version": "risk-model-0.1.0"

}
```

20. Development and Collaboration Workflow

20.1 Work Management

- Maintain one backlog in GitHub Projects (or Jira/Linear if already standard in the team).
- Every task gets an owner, priority and acceptance criteria.
- Keep the issue as the source of truth; avoid critical decisions living only in chat.
- Use short milestones/sprints with an end-of-sprint working demo.
- Track blockers explicitly.

20.2 Git Workflow

```
main(protected) | +-- feature/case-management+-- feature/prediction+--
feature/risk-map+-- feature/graph+-- feature/alerts
```

Feature branch -> commit -> push -> Pull Request -> review -> CI -> merge

20.3 Branch Rules

- No direct pushes to `main`.
- At least one reviewer for application changes.
- CI checks must pass before merge.
- Keep PRs small and focused.
- Link PRs to issues.
- Resolve review comments before merging.

20.4 Definition of a Feature

A feature is complete only when implementation, tests, documentation, error handling, review and integration are complete. "Code written" is not the definition of done.

21. Repository and Code Organization

cybercrime-intelligence-platform/

|-- frontend/

|-- backend/

|-- ml/

|-- intelligence/

|-- data-pipeline/

|-- database/

|-- infrastructure/

|-- docs/

|-- .github/

| |-- workflows/

| |-- CODEOWNERS

| `-- pull_request_template.md

`-- README.md

21.1 Documentation Tree

docs/

|-- requirements.md

|-- architecture.md

|-- data-dictionary.md

|-- database-erd.md

|-- api-spec.md

|-- ml-methodology.md

|-- security.md

|-- testing.md

|-- deployment.md

`-- demo-script.md

22. Team Responsibilities

Member / Role	Primary Ownership	Secondary Responsibility
1. Product / Tech Lead	Requirements, architecture, integration, technical decisions	DevOps, reviews, demo coordination
2. Data Engineer	Dataset, ETL, data quality, database/data pipeline	Data documentation

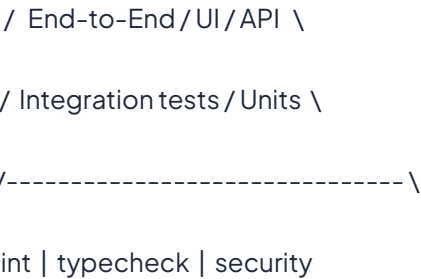
Member / Role	Primary Ownership	Secondary Responsibility
3. ML Engineer	Feature engineering, models, evaluation, inference	MLOps/model versioning
4. Intelligence Engineer	Graph analysis, risk engine, explainability	Analytical validation
5. Backend Engineer	APIs, auth, business logic, alert service	Security implementation
6. Frontend/GIS Engineer	Dashboard, GIS, investigator workspace	UX consistency

22.1 Ownership Rule

One owner is accountable for each module, but implementation is collaborative. Cross-module changes require communication through the documented API/data contract rather than informal assumptions.

23. Testing and Evaluation Strategy

23.1 Software Testing Pyramid



23.2 Required Test Categories

- Unit tests for data transformations, risk logic and backend services.
- API tests for authentication, validation, error cases and permissions.
- Integration tests for database + backend + ML prediction flow.

- End-to-end test of complaint -> prediction -> map -> alert.
- Security tests for access control, invalid input and secret handling.
- Performance checks on expected prototype workloads.

23.3 Acceptance Test: Golden Path

Create Case -> Save to DB -> Attach transaction data -> Run prediction -> Store prediction -> Display risk on map -> Show explanation -> Create high-risk alert -> Investigator acknowledges alert -> Record outcome

24. Security, Privacy and Governance

- Use role-based access control and enforce authorization at the backend, not only in the UI.
- Hash passwords using an approved password hashing mechanism; never store plaintext passwords.
- Use TLS/HTTPS for network traffic in deployed environments.
- Keep secrets in environment/secret management mechanisms, not source code.
- Mask or minimize personally identifiable information in the UI.
- Record audit events for sensitive actions such as case access, alert decisions and administrative changes.
- Validate all API inputs and use parameterized database access.
- Track third-party dependencies and patch known vulnerabilities.
- Label synthetic/demo data clearly and never imply access to restricted operational datasets without authorization.
- Treat predictions as decision-support signals rather than proof of criminal conduct.

Security Engineering Reference

The team will align its development practices with secure software development guidance such as NIST SSDF and application security guidance such as OWASP. Security is integrated throughout requirements, implementation, review, testing and release rather than treated as a final-stage checklist.

25. MLOps and Feedback Loop

Data version -> Train -> Evaluate -> Register model/version -> Deploy inference version -> Generate predictions -> Record outcomes -> Evaluate calibration / drift / false positives -> Retrain when justified -> Release new version

- Store model version with every prediction.
- Keep training code and configuration under version control.
- Separate training data from evaluation/test data.
- Avoid leakage from future events into training features.
- Track outcome labels when they become available.
- Review whether alert volume is operationally useful, not just whether the model metric improved.

26. Deployment Architecture

Developer machine -> Docker Compose + frontend + backend + PostgreSQL/PostGIS + ML service + optional Redis/worker -> CI pipeline (lint -> test -> build) -> Staging -> Demo / Production-like Environment

Environment Strategy

Environment	Purpose
Local	Individual development; synthetic data only.
CI	Automated tests and build validation.
Staging	Integrated environment for team testing and demo rehearsal.
Demo / Production-like	Stable presentation environment with controlled dataset and configuration.

27. MVP and Future Scope

27.1 MVP Release

- Authentication and roles
- Case management
- Synthetic data pipeline
- Transaction/network view
- Predictive model
- Risk score + explanation
- GIS heatmap
- Investigator dashboard
- Alert lifecycle
- Audit logs
- Dockerized deployment
- Automated tests and CI

27.2 Future Releases

- Real institutional integrations subject to authorization
- Streaming event ingestion
- AePS/CSP analytics
- Merchant/POS risk
- Advanced graph ML
- Automated model monitoring
- Large-scale cross-region coordination
- More sophisticated notification channels

28. Risks, Assumptions and Mitigations

Risk / Assumption	Impact	Mitigation
Limited real-world data	Model realism/performance may be weak	Use carefully designed synthetic/sanitized data; be transparent about limitations.

Risk / Assumption	Impact	Mitigation
Data leakage	Artificially inflated model performance	Use time-aware splits and leakage review.
Too many alerts	Users ignore alerts	Use ranked predictions, thresholds, severity and alert-volume monitoring.
Overly complex architecture	Slow development / integration failures	Use modular monolith + separate ML service; add complexity only when justified.
Unclear responsibilities	Parallel work conflicts	Module ownership, API contracts, PR review and task board.
Security defects	Sensitive-data exposure	Secure-by-design practices, RBAC, secrets management, tests and audit logging.
Prediction treated as certainty	Operational misuse	Display confidence, evidence and explicit decision-support labeling.

29. Demo Workflow

The final presentation should demonstrate one coherent story from detection to intervention rather than a collection of unrelated screens.

1. Investigator logs in
2. Dashboard shows active cases and current risk map
3. A new complaint/case is selected
4. Related transaction and entity data is displayed
5. Investigator runs prediction
6. System returns Top-K locations + time window
7. Highest-risk location becomes visible on GIS map
8. The investigator opens "Why is this high risk?"
9. Network/transaction evidence is displayed

10. High-risk alert is generated
11. Investigator acknowledges the alert
12. Outcome is recorded for future evaluation

Demo Rule: Never rely on a live external banking or government API for the core demo. Have a controlled synthetic dataset and a deterministic demo scenario so the end-to-end flow can be reproduced every time.

30. Deliverables and Presentation Plan

Deliverable	Contents
Engineering Blueprint	This document: requirements, architecture, workflow, roles, stack and standards.
System Architecture Diagram	High-level and component-level architecture.
ERD / Data Dictionary	Tables, fields, relationships, constraints and data definitions.
API Specification	Endpoints, request/response schemas, auth and error handling.
ML Report	Features, baselines, validation method, metrics, calibration and limitations.
Security Notes	RBAC, audit logging, secrets, dependency and input-validation approach.
Working Web App	Login, dashboard, GIS, case view, prediction, network and alerts.
Test Report	Software tests, ML evaluation and known limitations.
Deployment Package	Docker configuration, environment instructions and demo dataset.

Deliverable	Contents
PPT / Demo Script	Problem, gap, architecture, live flow, impact, limitations and future scope.

31. Definition of Done

- Task linked to an issue and assigned to an owner.
- Implementation follows project conventions.
- Automated tests added/updated where applicable.
- Input validation and error handling included.
- Documentation/API contract updated if behavior changed.
- Pull request reviewed by at least one teammate.
- CI checks pass.
- Merged into the protected main branch.
- Integrated feature tested in the staging/demo environment.
- Known limitations recorded.

32. Reference Standards and Engineering Principles

The following references are used as engineering guidance for this project. They are not claims that the prototype itself is compliant or production-certified.

Reference	URL	Use in Project
NIST Secure Software Development Framework (SSDF)	https://csrc.nist.gov/projects/ssdf	Secure software development practices integrated into the SDLC.
NIST SP 800–218 SSDF	https://csrc.nist.gov/pubs/sp/800/218/final	Secure software development framework reference.

Reference	URL	Use in Project
OWASP Application Security Verification Standard	https://owasp.org/www-project-application-security-verification-standard/	Application security requirements and verification reference.
OpenAPI Specification	https://spec.openapis.org/oas/latest.html	Machine-readable REST API contract reference.
Docker Compose documentation	https://docs.docker.com/compose/	Local/deployment container orchestration reference.
GitHub branch protection and pull requests	https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches	Collaborative code review and branch protection workflow.